

Project Report

Student Details:

Full Name - Vivek Kumar Mali

Roll Number - 24f2002719

Student Email : 24f2002719@ds.study.iitm.ac.in

Course - MAD-1 Project

Greeting everyone I am Vivek Kumar Mali, from Sirohi, Rajasthan, India. I am 20 years old and pursuing a BS degree in Data Science and Programming from IIT Madras , standalone. I am a passionate developer who enjoys solving real-world problems using Python and web technologies. I am especially interested in backend development and designing clean architectures.

Problem Statement:

Build a **web-based vehicle parking management system** that allows users to easily reserve parking spots and enables admins to manage locations, monitor usage, and view parking data analytics in real time.

Description:

This MAD-1 project aims to create a functional **vehicle parking management system** using Flask. It includes **user roles**, **parking lot/spot management**, and a **dashboard** to visualize parking activity and revenue trends. Users can book parking in advance, and admins can manage parking locations and users.

Technologies Used:

Backend:

- Python : A high-level programming language used to write server-side logic and backend controllers.
- Flask : A lightweight web framework for Python that provides routing, request handling, session management, and supports modular app structure via Blueprints.
- Flask Blueprints : Used to organize routes and controllers into reusable components like `auth_routes.py`, `routes.py`, and others.
- Flask SQLAlchemy : SQLAlchemy is a powerful ORM (Object Relational Mapper) that helps define database models as Python classes and manage complex relationships between them using Pythonic syntax.
- Dotenv (`python-dotenv`) : Loads environment variables from a `.env` file. Useful for hiding configuration details like secret keys and database URIs.

Frontend:

- HTML5 & Bootstrap (via CDN) : Used for page structure and styling.
- JavaScript : Enables client-side interactivity like form validation and dynamic chart rendering.

- Jinja2 : Flask's built-in templating engine that allows server-side rendering of HTML pages using dynamic variables and control flow.
- [Chart.js](#) : A flexible JavaScript charting library used to generate visual dashboards (bar graphs, pie charts, etc.) showing parking data like spot availability, usage trends, and revenue.

Database:

- SQLite : Lightweight, file-based relational database ideal for local development and small-scale apps. Easy to set up and does not require a separate server.

DB Schema Design:

➤ User(Many to One)

- user_id (PK) – Auto-incrementer
- user_email – Unique, required
- user_password – Required
- user_name, user_address, user_pincode – User info
- role_id (FK to Role.id) – User's access level

➤ Role(One to Many)

- id (PK) – Auto-incremented
- name – Unique role name (Admin/User)

➤ ParkingLot

- id (PK)
- prime_location_name, address, pincode – Lot details
- maximum_number_of_spots, price – Spot capacity & cost

➤ ParkingSpot

- id (PK)
- lot_id (FK to ParkingLot.id) – Belongs to lot
- status – Available / Reserved

➤ ReserveParkingSpot

- id (PK)
- spot_id, lot_id, user_id – FKs to respective models
- vehicle_no, address, pincode – Booking details
- parking_timestamp, leaving_timestamp – Time info
- parking_cost – Cost for reservation

API Design

- Role assignment
- Parking lot creation and listing
- Spot availability checks
- Reservation (create, update, cancel)

Architecture and Features

The project follows a modular MVC-style structure:

- Controllers: Business logic in `controller/routes.py` and `auth_routes.py`
- Database: initialize DB on `controller/database.py`
- Models: Defined in `controller/models.py`
- Templates: All Jinja2 HTML files in `/templates/`
- Static: Contains CSS, JS, and images
- Main App Entry: `main.py` initializes Flask

Features Implemented:

This project includes all core functionalities required for a complete Vehicle Parking Management System, along with enhancements like charts and user roles. Here's a detailed breakdown.

User & Role Management:

- User Registration & Login:
 - Users can sign up with their name, email, password, and address, pincode. After logging in, the system verifies their role to direct them appropriately.
- Role-Based Access Control (RBAC):
 - Admin users have access to all management tools like adding parking lots and viewing analytics.
 - Regular users can search, reserve, and view their bookings.

Parking Lot & Spot Management:

- Admin Dashboard:
Admins can:
 - Add new parking lots with name, address, capacity, and pricing.
 - View all available/reserved spots.
 - Track revenue per lot.
- Parking Spot Tracking:
 - Each spot has a real-time `status` (Available, Occupied).
 - Spots are associated with specific lots and updated on each reservation.

Reservation System:

- Spot Reservation Flow:
Users can:
 - Choose a lot
 - View spot availability
 - Reserve a spot by entering vehicle number and address
 - Select entry and leaving time
- Cost Calculation:
 - Reservation cost is automatically calculated based on the lot's pricing and duration.
- Booking History:
 - Users can view their current and past reservations with timestamps, cost, and location.

Dashboard & Analytics

- Admin Chart Dashboard (Chart.js):
Interactive visualizations include:
 - Parking spot availability across lots
 - Revenue trends over time
 - Booking activity and usage statistics
- Real-Time Data Updates:
 - Charts and tables reflect changes as users make reservations or as lots are updated.

User Interface

- Responsive UI with Bootstrap (via CDN):
 - The interface is styled using Bootstrap 5 for consistency and responsiveness.
 - Layouts adapt well to desktop and mobile screens.
- Clean HTML/CSS with Jinja2 Templates:
 - Jinja2 makes templates dynamic, rendering user-specific data in views like dashboards and reservation history.

Persistence & Configuration

- SQLite Database
 - Used for quick setup and development — stores all users, roles, lots, spots, and reservations.
- .env Configuration
 - All sensitive settings like secret keys and DB URI are managed via a `.env` file for security and flexibility.

ER Diagram:



AI Percentage :

- I used 4% AI (ChatGPT) in Bootstrap/CSS for Styling and layout
- I used 5% AI to resolve errors.

Video Link: 📺 **MAD-1 Project Vehicle Parking App.mp4**